Designing Data Pipelines That Don’t Hate You Six Months Later

A Sci-Fi Kung Fu Inspired Story About Data Engineering Fundamentals





MOMENTUM
Developer Conference
powered by The Circuit



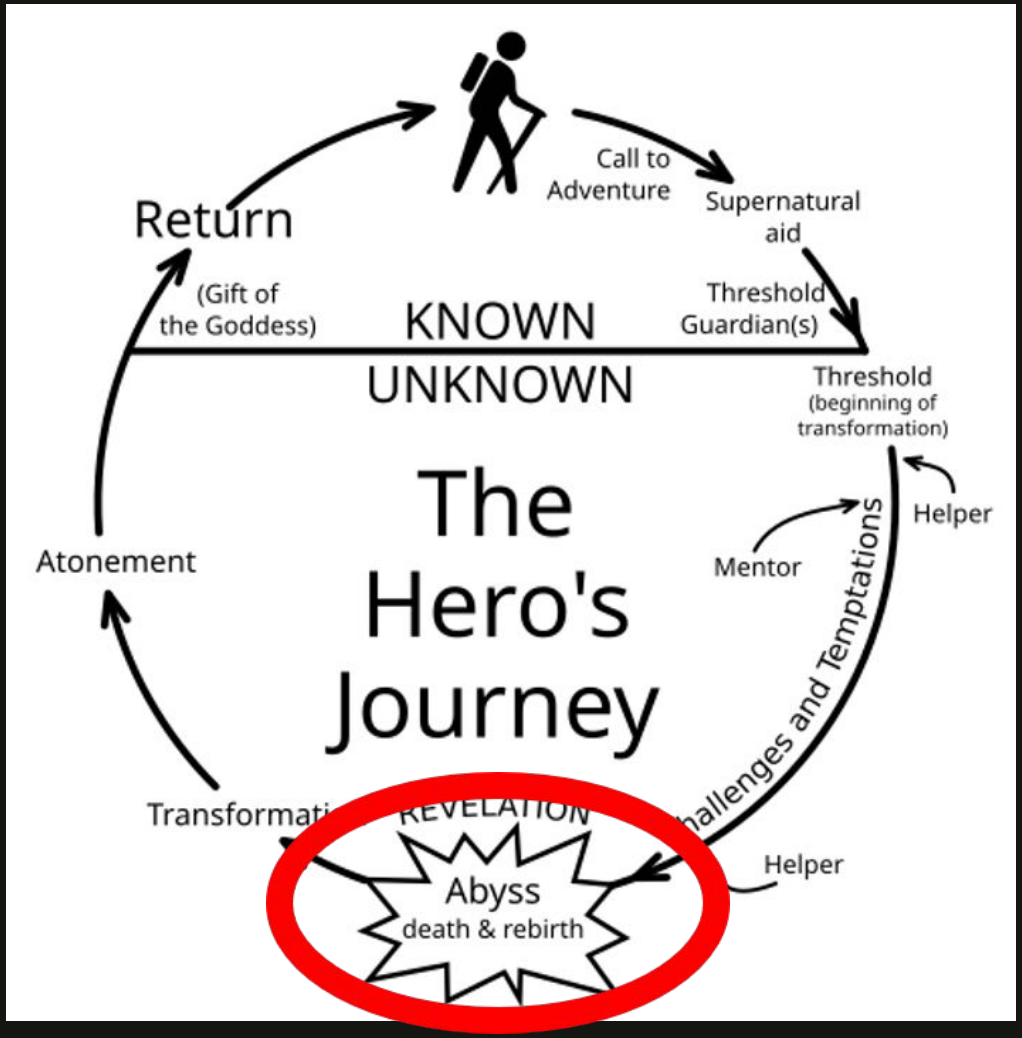
ABOUT ME




Chris, Cartographer of Chaos 2
Legendary Creature — Human Shapeshifter
Haste, Trample
When Chris enters, create a Map token and a Clue token.
: Scry 2, then draw a card. If you do, discard a card.
Whenever you cast a spell that targets a permanent, investigate.
At your end step, if you control a Map token
• Build the Path — Return a permanent card from your graveyard to your hand.
At your end step, if you control a Clue token
• Translate the Terrain — Draw a card, each opponent discards a card.
Hand me the Chaos. I'll bring back the map,
the path through, and what's in the way.
3/5
EDO • PA DS&D • 001/001 • R • Chris Birie
CUSTOM FAN CARD • NOT FOR SALE


VT
WING TSUN
詠春

Data Pipelines! Specifically great data pipelines that don't hate you six months later!


Call to
Adventure
Supernatural
aid
Threshold
Guardian(s)
Threshold
(beginning of
transformation)
Helper
Mentor
Challenges and Temptations
Helper
REVELATION
Abyss
death & rebirth
Transformati
Atonement
Return
(Gift of
the Goddess)
KNOWN
UNKNOWN
The
Hero's
Journey


THE LAST STARFIGHTER
The adventure of a lifetime is about to begin.
INCEPTION
THE GOONIES
MATRIX
GHOSTBUSTERS
INTERSTELLAR
Galaxy Quest
3:00 AM
[ERROR] Schema mismatch: expected 'customer_id' but found 'cust_id'
[ERROR] Missing required column: 'account_status'
[ERROR] Type mismatch: expected 'created_at' as TIMESTAMP, received STRING
[ERROR] Downstream model out of sync with source schema
[ERROR] Duplicate records detected for run_id=2026-04-28
[ERROR] Primary key violation: record already exists
[WARN] Retry produced different row count than original run
[ERROR] Merge failed: duplicate keys found in source batch
[ERROR] Pipeline failed with exit code 1
[WARN] No metrics emitted for job duration, row count, or error count
[WARN] No correlation_id found for failed batch
[ERROR] Alert triggered, but failing step is unknown
[ERROR] Production validation failed: unexpected null rate in customer_id
[ERROR] Transformation output does not match expected contract
[ERROR] Deprecated field still referenced: legacy_customer_id
[ERROR] Contract violation: downstream expects one row per customer
[WARN] Business rule changed but transformation logic was not updated
[ERROR] Reporting layer incompatible with new aggregation grain
#data-pipeline-alerts
25
Sarah 2:59 AM
Did the upstream team change the payload again?
Mike 2:59 AM
Can we safely rerun this job or will it double-load everything?
Priya 2:59 AM
Where did it fail? Do we have logs for that?
Alex 3:00 AM
How did this pass QA?
Jordan 3:00 AM
That's not how the process works anymore.
Message #data-pipeline-alerts
Boss
mobile
7 Missed Calls
78 Messages
What would Cooper do?
MAYBE NOTHING IS REAL
FOCUS COMMITMENT SHEER WILL
DON'T TRUST THEY ANYONE
Goonies never say die.
NEED MORE COFFEE
OCT 26 1985 01:21 AM
THE LAST STARFIGHTER
GALAXY QUEST
THE GOONIES



[Freeze Frame]
Record Scratch

You may be wondering how I got here...

SIX MONTHS EARLIER...



Enter the ~~Dragon~~ Bruce.



Bruce was brilliant.

Bruce was fast.

Bruce probably used AI.

Bruce shipped early.

The pipeline worked beautifully

Everyone was happy.

Bruce got a pat on the back.

And probably a promotion.

This was Bruce’s pipeline.

It is now your problem.

Pipelines built on shaky fundamentals don’t fail on day one.

They fail when pressure arrives.

BRUCE DIDN'T BUILD A BAD PIPELINE.

He built a pipeline that worked perfectly for the fight he trained for.

The problem is the fight changed.

The pipeline didn't fail because Bruce was lazy or bad at his job.
It failed because nobody taught him the fundamentals, or they were ignored.
And then Bruce was gone.

Bruce is gone. But his pipeline lives on.

And right now, at 3am, it is absolutely on fire.

WHAT HAPPENED?

Six months ago, Bruce was building the pipeline. He didn't know what was coming. Nobody did.

Every lesson ahead is one Bruce never learned.
Each principle — a mistake he made so you don't have to.

Four lessons. Four chances to change the outcome.

MAIN LESSON

DESIGN FOR CHANGE

“Be water, my friend”

BRUCE'S MISTAKE

6 months ago: Bruce’s pipeline was tight, clean, and exactly right for the problem in front of him. It handled the volume, the schema, the delivery cadence. It was built for the fight he knew was coming.

Since Then: Requirements shifted. The source system restructured. The business wants to backfill 18 months of history.

Result: The pipeline didn’t bend. It shattered.

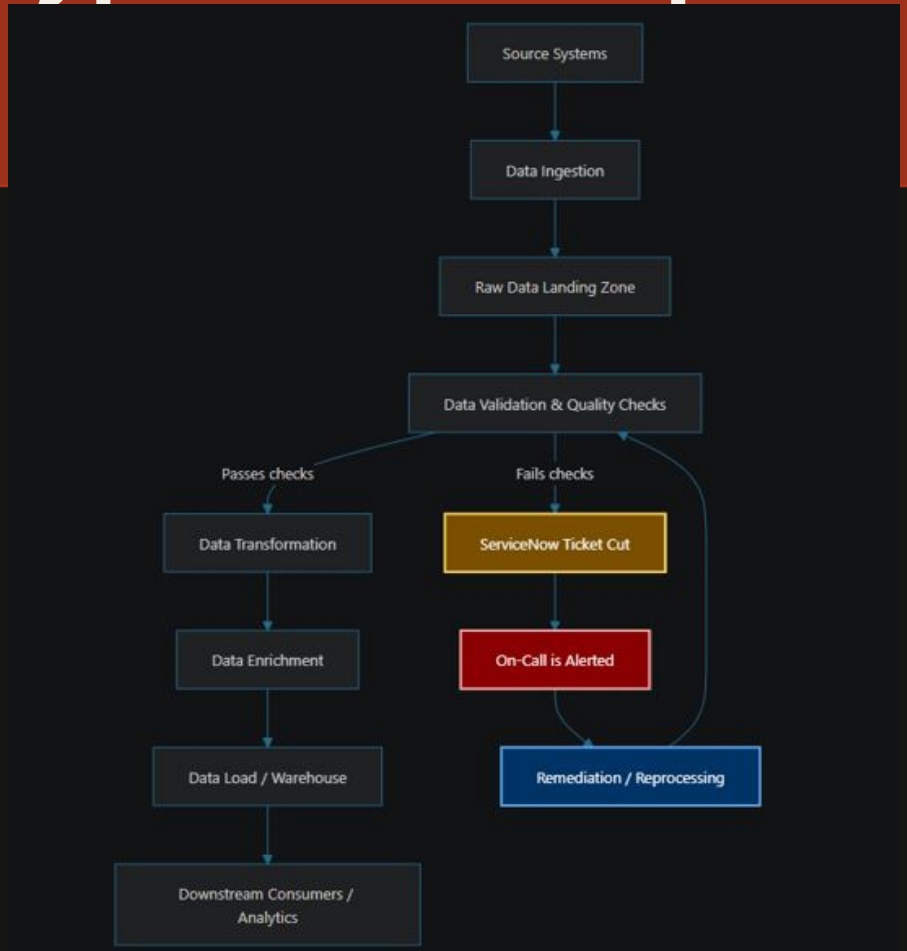
THE LESSON

Water doesn’t resist the container. It adapts. Bruce’s pipeline was concrete, perfectly shaped for the original mold.

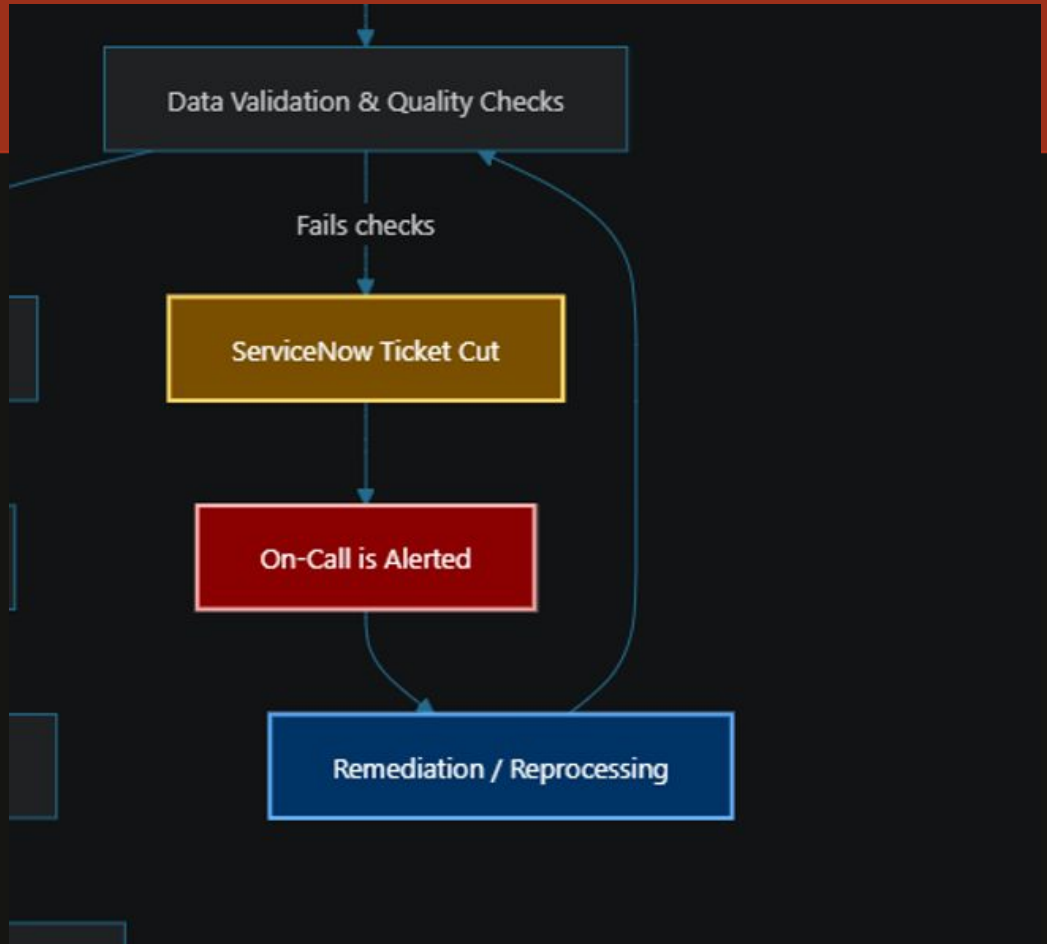
When the mold changed, things broke

"Volume is the easy scaling problem. Changing requirements is the real test."

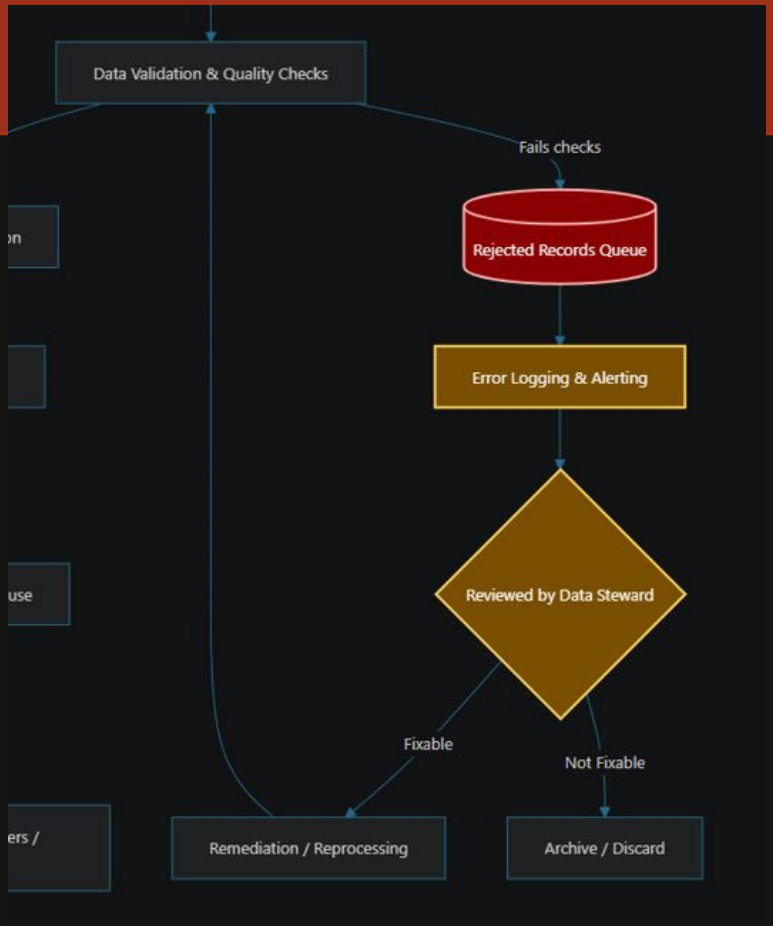
Typical Data Pipeline



Typical Data Pipeline



Fix 1: Rejected Records Queue



THE FOUR LESSONS BRUCE NEVER LEARNED

01

SCHEMA EVOLUTION

Absorb what is useful, reject what is useless

02

IDEMPOTENCY

Strike twice, Leave one mark

03

OBSERVABILITY

You cannot escape the blow you never saw coming

04

TESTABILITY

Train the move before the fight, not during it

"Do not build for the blow you are taking. Build for the blow you have not yet seen..."

— Some Wise Sifu (Probably)


3:00 AM
[ERROR] Schema mismatch: expected 'customer_id' but found 'cust_id'
[ERROR] Missing required column: 'account_status'
[ERROR] Type mismatch: expected 'created_at' as TIMESTAMP, received STRING
[ERROR] Downstream model out of sync with source schema
[ERROR] Duplicate records detected for run_id=2026-04-28
[ERROR] Primary key violation: record already exists
[WARN] Retry produced different row count than original run
[ERROR] Merge failed: duplicate keys found in source batch
[ERROR] Pipeline failed with exit code 1
[WARN] No metrics emitted for job duration, row count, or error count
[WARN] No correlation_id found for failed batch
[ERROR] Alert triggered, but failing step is unknown
[ERROR] Production validation failed: unexpected null rate in customer_id
[ERROR] Transformation output does not match expected contract
[ERROR] Deprecated field still referenced: legacy_customer_id
[ERROR] Contract violation: downstream expects one row per customer
[WARN] Business rule changed but transformation logic was not updated
[ERROR] Reporting layer incompatible with new aggregation grain
#data-pipeline-alerts
25
Sarah 2:59 AM
Did the upstream team change the payload again?
Mike 2:59 AM
Can we safely rerun this job or will it double-load everything?
Priya 2:59 AM
Where did it fail? Do we have logs for that?
Alex 3:00 AM
How did this pass QA?
Jordan 3:00 AM
That's not how the process works anymore.
Message #data-pipeline-alerts
Boss
mobile
7 Missed Calls
78 Messages
OCT 26 1985 01:21 AM


THE LAST STARFIGHTER
INCEPTION
THE GOONIES
OUTATIME
MATRIX
TWIN PINES MALL
1:16 AM
BACK TO THE FUTURE
GHOSTBUSTERS
GALAXY QUEST
I CODA
THEREFORE
I CAFFEINATE
NASA
STARFIGHTER
DATA
>
SLEEP

WHAT IF...?



LESSON 01

SCHEMA EVOLUTION

Absorb what is useful, reject what is useless

BRUCE'S MISTAKE

Blindly trusting the source to never change

The rigid form breaks under the blow it did not expect

THE LESSON

Notice that the stiffest tree is most easily cracked, while the bamboo or willow survives by bending with the wind

The living form bends, absorbs, and remains.

Make unbreakable only what must never move, the rest must be free to grow, or it will only break

"Your pipeline will outlive the person who designed the source system. Plan for that."

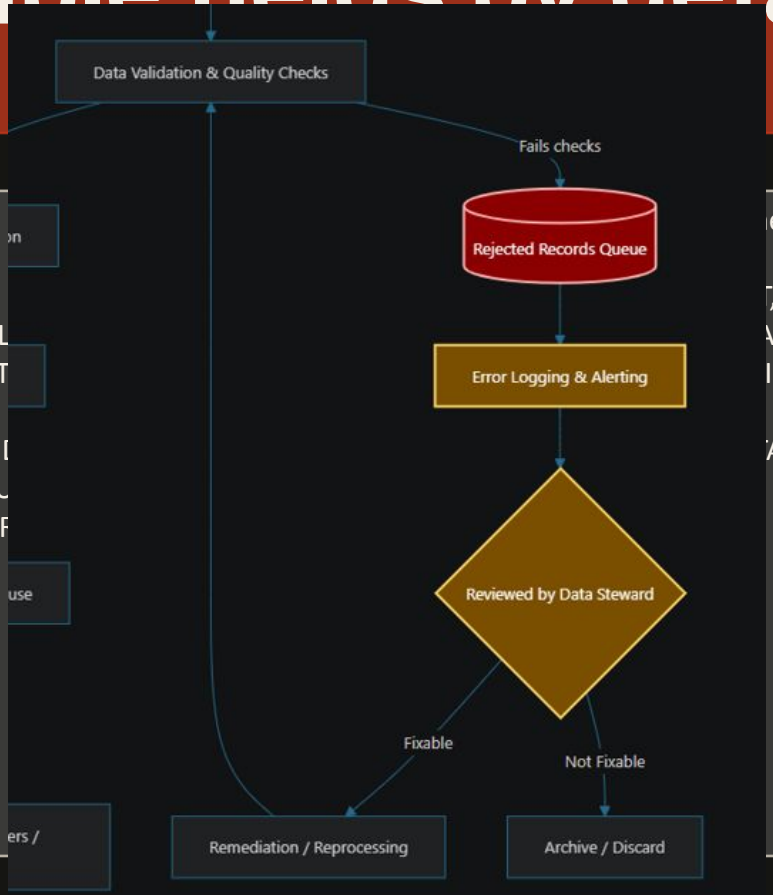
Fix 1: Nullable fields by default

```
CREATE TABLE customer_event (
    event_id BIGINT NOT NULL,
    customer_id BIGINT NOT NULL,
    event_type VARCHAR(50) NOT NULL,
    event_timestamp TIMESTAMP    NOT NULL,
    payload JSONB NOT NULL,
    ingested_at TIMESTAMP NOT NULL DEFAULT
                                        CURRENT_TIMESTAMP,
    CONSTRAINT pk_customer_event PRIMARY KEY (event_id)
);
```

```
CREATE TABLE customer_event (
    event_id BIGINT,
    customer_id BIGINT,
    event_type VARCHAR(50),
    event_timestamp TIMESTAMP,
    payload JSONB,
    ingested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Fix 1: Nullable fields by default

```
CREATE TABLE customer_event (
    event_id BIGINT NOT NULL,
    customer_id BIGINT NOT NULL,
    event_type VARCHAR(50) NOT NULL
    event_timestamp TIMESTAMP   NOT
    payload JSONB NOT NULL,
    ingested_at TIMESTAMP NOT NULL D
                                        CU
    CONSTRAINT pk_customer_event PR
);
```




```
er_event (

,
AR(50),
IMESTAMP,

AMP DEFAULT CURRENT_TIMESTAMP
```

Fix 2: Versioned Schema Contracts
Versioned Contracts

```
// customer_event_v1.avsc
{
  "type": "record",
  "name": "CustomerEvent",
  "fields": [
    {"name": "cust_id", "type": "string"},
    {"name": "event_ts", "type": "long"},
    {"name": "amount", "type": "double"}
  ]
}
```

```
// customer_event_v2.avsc
{
  "type": "record",
  "name": "CustomerEvent",
  "fields": [
    {"name": "customer_id", "type": "string"},
    {"name": "event_ts", "type": "long"},
    {"name": "amount", "type": "double"}
  ]
}
```

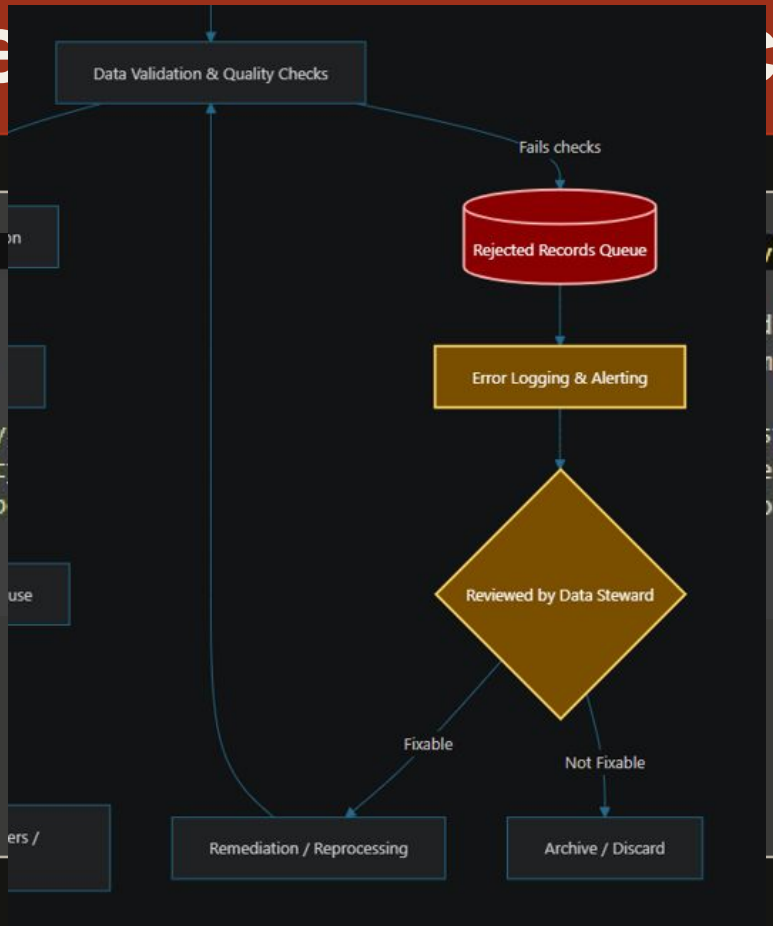
Fix 2: Versioned Schema Contracts Allow Both, Deprecate Old

```
on incoming(batch):
    incoming_schema = describe(batch)
    version         = registry.identify(incoming_schema)
    if version == v1:
        records = read(batch, v1_schema)
        records = rename(records, cust_id -> customer_id)   # adapter
    else if version == v2:
        records = read(batch, v2_schema)
    write_downstream(records)
```

Deprecate the old schema, Alert the producers!

Fix 2: Versioned Schema Contracts

```
// customer_event_v1.avsc
{
  "type": "record",
  "name": "CustomerEvent",
  "fields": [
    {"name": "cust_id", "ty
    {"name": "event_ts", "t
    {"name": "amount", "typ
  ]
}
```




```
/2.avsc
",
erEvent",

tomer_id", "type": "string"},
nt_ts", "type": "long"},
unt", "type": "double"}
```

Fix 3: Defensive parsing

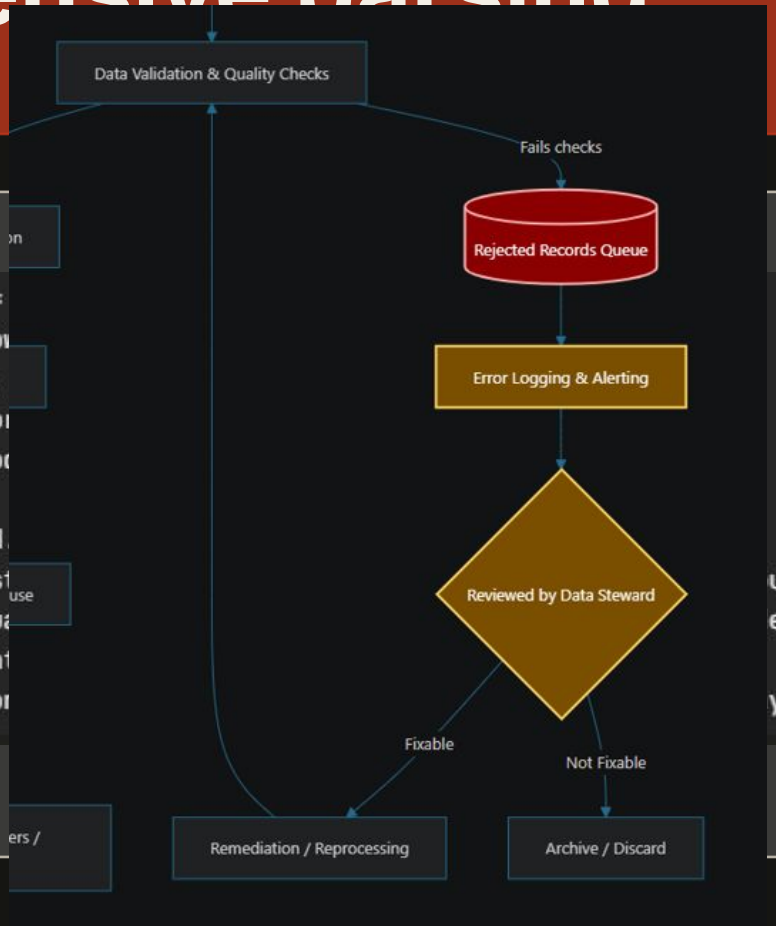
```
# Naive — one bad record kills the whole batch
parse(row):
    return {
        customer_id : row.customer_id,
        amount      : as_number(row.amount),
        event_ts    : as_timestamp(row.event_ts),
    }
```

```
# Defensive — bad records get quarantined, batch keeps moving
parse(row):
    try:
        validate row against CustomerEvent shape:
            customer_id : string, required
            amount      : number, >= 0
            event_ts    : timestamp, required
        # unknown fields are ignored, not fatal
        return (clean_record, no_error)
    on validation_failure as err:
        return (no_record, { row: row, reason: err })
```

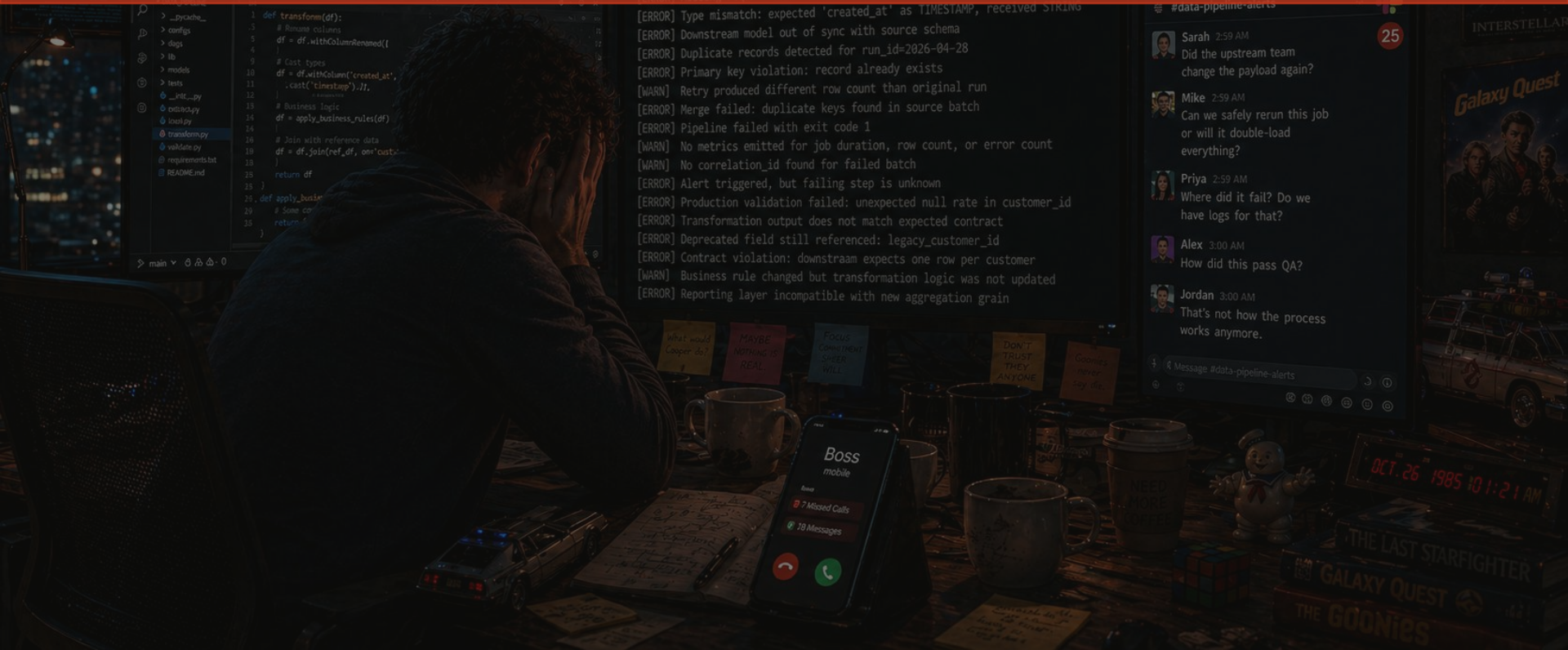
Fix 3: Defensive parsing

```
good, bad = [], []
for each row in batch:
    record, err = parse(row)
    if record:
        good.append(record)
    else:
        bad.append(err)
write_downstream(good)              # 9,997 records flow through
write_to_quarantine(bad)            # 3 records land in a side table
if bad.count / batch.count > threshold:
    alert_oncall()                  # page someone if too many failures
```

Fix 3: Defensive parsing



THE PAST HAS CHANGED!




THE LAST STARFIGHTER
INCEPTION
THE GOONIES
DAILY PROPHET
THE BOY WHO LIES?
MATRIX
GHOSTBUSTERS
INTERSTELLAR
Galaxy Quest
EXPLORER
DATA_PIPELINE
transform.py
schema_utils.py
load.py
def transform(df):
Rename columns
df = df.withColumnRenamed({
}
Cast types
df = df.withColumn('created_at',
:cast("timestamp")).
}
Business logic
df = apply_business_rules(df)
}
Join with reference data
df = df.join(ref_df, on="cust
}
return df
}
def apply_business_r
Some complex
return df
}
main
3:00 AM
[ERROR] Duplicate records detected for run_id=2026-04-28
[ERROR] Primary key violation: record already exists
[WARN] Retry produced different row count than original run
[ERROR] Merge failed: duplicate keys found in source batch
[ERROR] Pipeline failed with exit code 1
[WARN] No metrics emitted for job duration, row count, or error count
[WARN] No correlation_id found for failed batch
[ERROR] Alert triggered, but failing step is unknown
[ERROR] Production validation failed: unexpected null rate in customer_id
[ERROR] Transformation output does not match expected contract
[ERROR] Deprecated field still referenced: legacy_customer_id
[ERROR] Contract violation: downstraam expects one row per customer
[WARN] Business rule changed but transformation logic was not updated
[ERROR] Reporting layer incompatible with new aggregation grain
#data-pipeline-alerts
20
Mike 2:59 AM
Can we safely rerun this job or will it double-load everything?
Priya 2:59 AM
Where did it fail? Do we have logs for that?
Alex 3:00 AM
How did this pass QA?
Jordan 3:00 AM
That's not how the process works anymore.
Message #data-pipeline-alerts
What would Cooper do?
MAYBE NOTHING IS REAL
FOCUS COMMITMENT SHEER WILL
DON'T TRUST THEY ANYONE
Goonies never say die.
Boss
mobile
5 Missed Calls
15 Messages
NEED MORE COFFEE
OCT 26 1985 01:21 AM
THE LAST STARFIGHTER
GALAXY QUEST
THE GOONIES


THE LAST STARFIGHTER
INCEPTION
THE GOONIES
OUTATIME
MATRIX
BACK TO THE FUTURE
GHOSTBUSTERS
INTERSTELLAR
GALAXY QUEST
I CODA THEREFORE I CAFFEINATE
NASA
STARFIGHTER
DATA > SLEEP

WHAT IF...?



LESSON 02

IDEMPOTENCY

Strike twice. Leave one mark

BRUCE'S MISTAKE

The pipeline will run exactly once per day, no more, no less with no duplicate data

The fighter who trains for one clean exchange is not prepared for the real fight.

THE LESSON

Expect to be hit more than once

Real fights are messy even if they start out clean

"If running it twice breaks it, it was already broken."

Fix 1: Deterministic Keys

```
for record in batch:
    record.id = new_uuid()
    write(record)
# Run twice → two rows, two different ids,
# same business event.
```

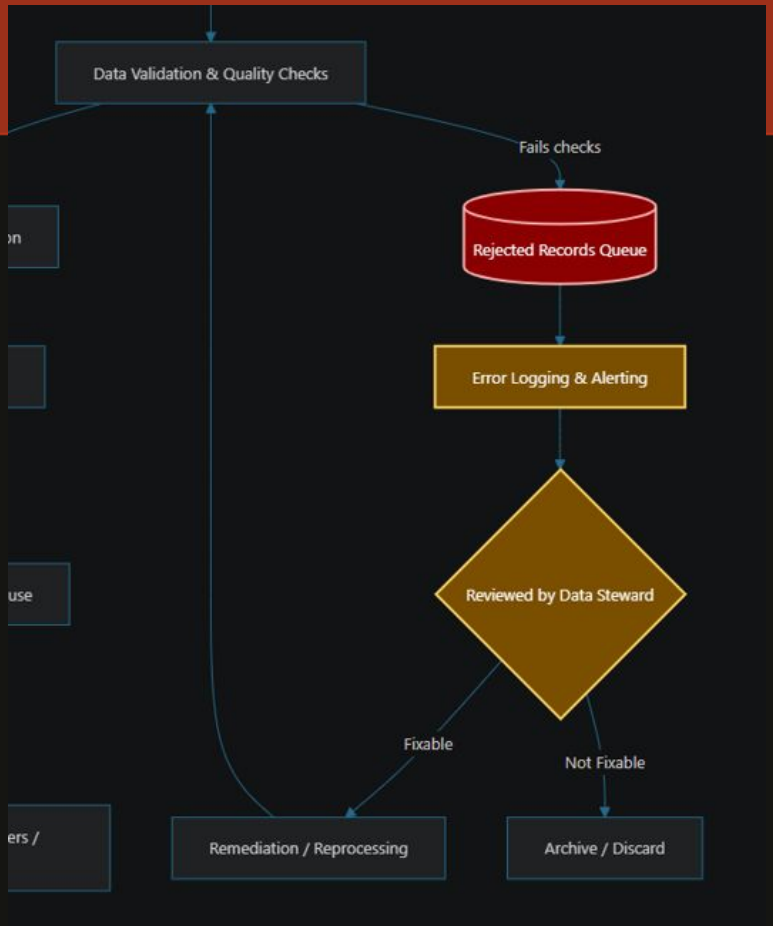
```
for record in batch:
    record.id = hash(
        record.customer_id,
        record.event_ts,
        record.amount
    )
    upsert(record)
# Run twice → same id both times.
# Second run is a no-op or an update.
```

Fix 2: Upsert Over Insert

```
for record in batch:
    INSERT INTO customer_event
        (id, customer_id, amount, event_ts)
    VALUES
        (record.id, record.customer_id,
         record.amount, record.event_ts)
# Run twice → duplicates.
# Pipeline failure mid-batch → partial dupes.
```

```
for record in batch:
    UPSERT INTO customer_event
        (id, customer_id, amount, event_ts)
    VALUES
        (record.id, record.customer_id,
         record.amount, record.event_ts)
    ON CONFLICT (id) DO UPDATE
        SET amount   = excluded.amount,
            event_ts = excluded.event_ts
# Run twice → same result.
# Mid-batch failure → safe to retry.
```

Rejected Records Queue



THE PAST HAS CHANGED!




3:00 AM
[ERROR] Pipeline failed with exit code 1
[WARN] No metrics emitted for job duration, row count, or error count
[WARN] No correlation_id found for failed batch
[ERROR] Alert triggered, but failing step is unknown
[ERROR] Production validation failed: unexpected null rate in customer_id
[ERROR] Transformation output does not match expected contract
[WARN] No unit tests found for transform.py
[WARN] No fixture data available for this edge case
[ERROR] Deprecated field still referenced: legacy_customer_id
[ERROR] Contract violation: downstream expects one row per customer
[WARN] Business rule changed but transformation logic was not updated
[ERROR] Reporting layer incompatible with new aggregation grain
#data-pipeline-alerts
15
Priya 2:59 AM
Where did it fail? Do we have logs for that?
Alex 3:00 AM
How did this pass QA?
Jordan 3:00 AM
That's not how the process works anymore.
Message #data-pipeline-alerts
Boss
mobile
3 Missed Calls
10 Messages
What would Cooper do?
MAYBE NOTHING IS REAL
FOCUS COMMITMENT SHEER WILL
DON'T TRUST THEY ANYONE
Goonies never say die.
NEED MORE COFFEE
It's not a bug it's an extreme feature.
OCT 26 1985 01:21 AM
THE LAST STARFIGHTER
GALAXY QUEST
THE GOONIES


THE LAST STARFIGHTER
INCEPTION
THE GOONIES
OUTATIME
MATRIX
Coors LIGHT
BACK TO THE FUTURE
GHOSTBUSTERS
INTERSTELLAR
GALAXY QUEST
I CODA THEREFORE I CAFFEINATE
NASA
STARFIGHTER
DATA > SLEEP

WHAT IF...?



LESSON 03

OBSERVABILITY

You cannot escape the blow you never saw coming

BRUCE'S MISTAKE

6 Months Ago: The data was flowing correctly, there isn't a need to log beyond basic error handling.

Today: Many non fatal errors didn't kill the pipeline nor did it let anyone know about those errors

Result: We have no idea how long this has been happening or when to start looking

THE LESSON

The master fighter does not move faster than the novice fighter - they see sooner, they can anticipate what is coming next by knowing tendencies, they can feel what is coming next

"Hope is not a monitoring strategy."

Fix 1: Row Counts at Every Stage

```
records = extract(source)
records = transform_a(records)
records = transform_b(records)
records = transform_c(records)
load(target, records)
# 50k in. 45k out. Nobody noticed.
```

```
records = extract(source)
emit_metric("extract.count", count(records))
records = transform_a(records)
emit_metric("transform_a.count", count(records))
records = transform_b(records)
emit_metric("transform_b.count", count(records))
records = transform_c(records)
emit_metric("transform_c.count", count(records))
load(target, records)
emit_metric("load.count", count(records))
# Every drop is visible. Every drift is graphable.
```

Fix 2: Null Rate Tracking

```
records = extract(source)
load(target, records)
# customer_email goes 2% null -> 40% null overnight.
# Nobody knows until a report breaks next week.
```

```
records = extract(source)
for field in key_fields:
    null_rate = count_where(records, field is null) / count(records)
    emit_metric("null_rate." + field, null_rate)
    if null_rate > baseline[field] * tolerance:
        alert_oncall("null rate spiked on " + field)
load(target, records)
# Upstream change shows up in the metric before it shows up in a complaint.
```

Fix 3: Latency Monitoring

```
run_pipeline()
# Usually 12 minutes. Today 47.
# Found out when the dashboard was late.
```

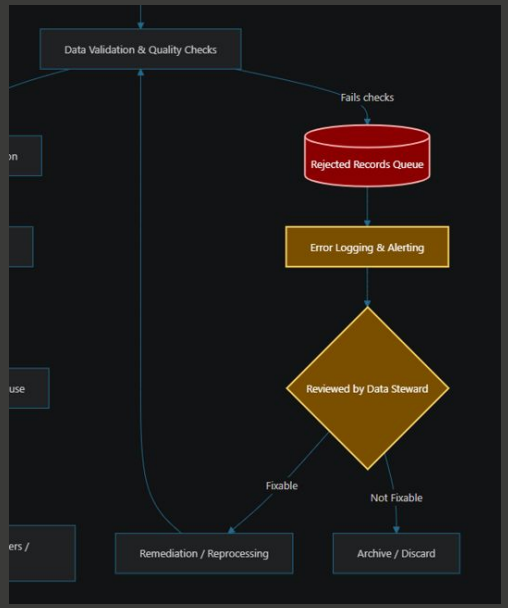
```
start = now()
run_pipeline()
duration = now() - start
emit_metric("pipeline.duration", duration)
rolling_avg = read_metric("pipeline.duration", window="7d")
if duration > rolling_avg * 2:
    alert_oncall("pipeline ran " + duration + ", avg is " + rolling_avg)
# You hear about the slow run before the business does.
```

Fix 4: Alert on Anomalies, Not just Exceptions

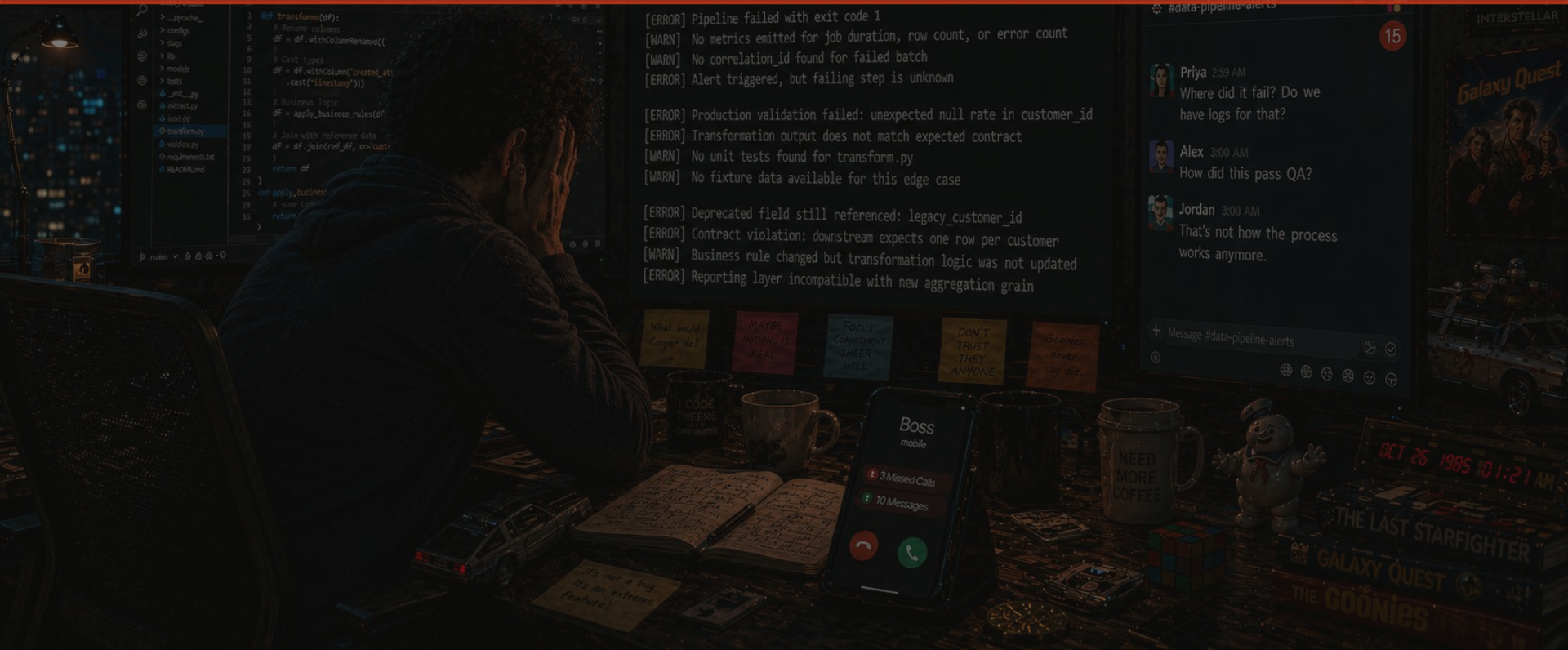
```
try:
    run_pipeline()
except Exception as e:
    alert_oncall(e)
# Source sent zero records. Pipeline ran cleanly.
# No exception, no alert, no data. Silent zero.
```

Fix 4: Alert on Anomalies, Not just Exceptions

```
try:
    run_pipeline()
except Exception as e:
    alert_oncall(e)
# Source sent zero records. Pipeline ran cleanly.
# No exception, no alert, no data. Silent zero.
```



THE PAST HAS CHANGED!




3:00 AM
[ERROR] Production validation failed: unexpected null rate in customer_id
[ERROR] Transformation output does not match expected contract
[WARN] No unit tests found for transform.py
[WARN] No fixture data available for this edge case
[ERROR] Deprecated field still referenced: legacy_customer_id
[ERROR] Contract violation: downstream expects one row per customer
[WARN] Business rule changed but transformation logic was not updated
[ERROR] Reporting layer incompatible with new aggregation grain
#data-pipeline-alerts
10
Alex 3:00 AM
How did this pass QA?
Jordan 3:00 AM
That's not how the process works anymore.
Sam 3:01 AM
Can we get an ETA on a fix?
Message #data-pipeline-alerts
Boss
mobile
2 Missed Calls
7 Messages
What would Cooper do?
MAYBE NOTHING IS REAL
Focus Commitment SHEER WILL
DON'T TRUST THEY ANYONE
Goonies never say die.
NEED MORE COFFEE
It's not a bug it's an extreme feature.
THE LAST STARFIGHTER
GALAXY QUEST
THE GOONIES
INCEPTION
THE GOONIES
MATRIX
GHOSTBUSTERS
INTERSTELLAR
Galaxy Quest


THE LAST STARFIGHTER
INCEPTION
THE GOONIES
OUTATIME
PHONE
BACK TO THE FUTURE
GHOSTBUSTERS
INTERSTELLAR
GALAXY QUEST
MATRIX
I CODA THEREFORE I CAFFEINATE
NASA
DATA > SLEEP

WHAT IF...?



LESSON 04

TESTABILITY

Train the move before the fight, not during it

BRUCE'S MISTAKE

6 months ago: Bruce built the pipeline and tested it in production. It worked. Data came in, data went out. Looked right. He shipped it.

Today: A bug was found — records with null values in a key field were being silently dropped. The only way to fix it was to run the corrected version in production and watch what happened.

Result: Every change is a live gamble. Every fix introduces new risk. Nobody has confidence in anything because nothing has ever been tested outside of the real environment.

THE LESSON

You do not rise to the moment; you fall to your preparation

You don’t find out if a technique works in a real fight. You already know — because you ran it a thousand times in the gym.

"Train like you would fight. Don't fight to find out if you trained."

Fix 1: Pure Transformation Functions

```
def run():
    conn = connect(prod_db)
    rows = conn.query("SELECT * FROM raw_customer_event")
    cleaned = []
    for row in rows:
        row.customer_id = row.customer_id.lower()
        row.amount      = row.amount or 0
        if row.amount >= 0:
            cleaned.append(row)
    conn.execute("INSERT INTO customer_event ...", cleaned)
# Can't test the transform without a live database.
```

```
# Pure — input in, output out, no side effects
def clean(records):
    out = []
    for row in records:
        row.customer_id = row.customer_id.lower()
        row.amount      = row.amount or 0
        if row.amount >= 0:
            out.append(row)
    return out
# Orchestration is the thin shell around the pure core
def run():
    rows    = extract(source)
    cleaned = clean(rows)
    load(target, cleaned)
# clean() is unit-testable with a list of 5 dicts.
```

Fix 2: Parameterized Inputs

```
def run():
    conn = connect("prod-db.company.internal")
    rows = conn.query(
        "SELECT * FROM raw_customer_event "
        "WHERE event_date = '2026-05-28'"
    )
    conn.execute("INSERT INTO prod.customer_event ...", rows)
# Hardwired to prod. Can't safely run anywhere else.
```

```
def run(env, source, target, window):
    rows = extract(source, where=window)
    load(target, rows)
# Same function, different parameters
run(env="prod",    source=prod_src,    target=prod_tgt,    window="2026-05-28")
run(env="staging", source=staging_src, target=staging_tgt, window="2026-05-28")
run(env="test",    source=fixture_src, target=test_tgt,    window="2026-05-28")
# The pipeline is a function of its parameters.
```

Fix 3: Fixture-Based Unit Tests

```
# "Testing" = run the job in prod, eyeball the output, hope.
# Last month: records with null created_at silently dropped.
# Fixed it. No test. Could regress tomorrow and we'd never know.
```

```
fixture_nulls = [
    { customer_id: "A", created_at: null, amount: 10 },
    { customer_id: "B", created_at: null, amount: 20 },
    { customer_id: "C", created_at: null, amount: 30 },
]
def test_null_created_at_gets_defaulted():
    result = clean(fixture_nulls)
    assert count(result) == 3                          # none dropped
    for row in result:
        assert row.created_at == DEFAULT_TIMESTAMP     # all defaulted
# Every production incident becomes a fixture.
# Every fixture becomes permanent regression coverage.
```

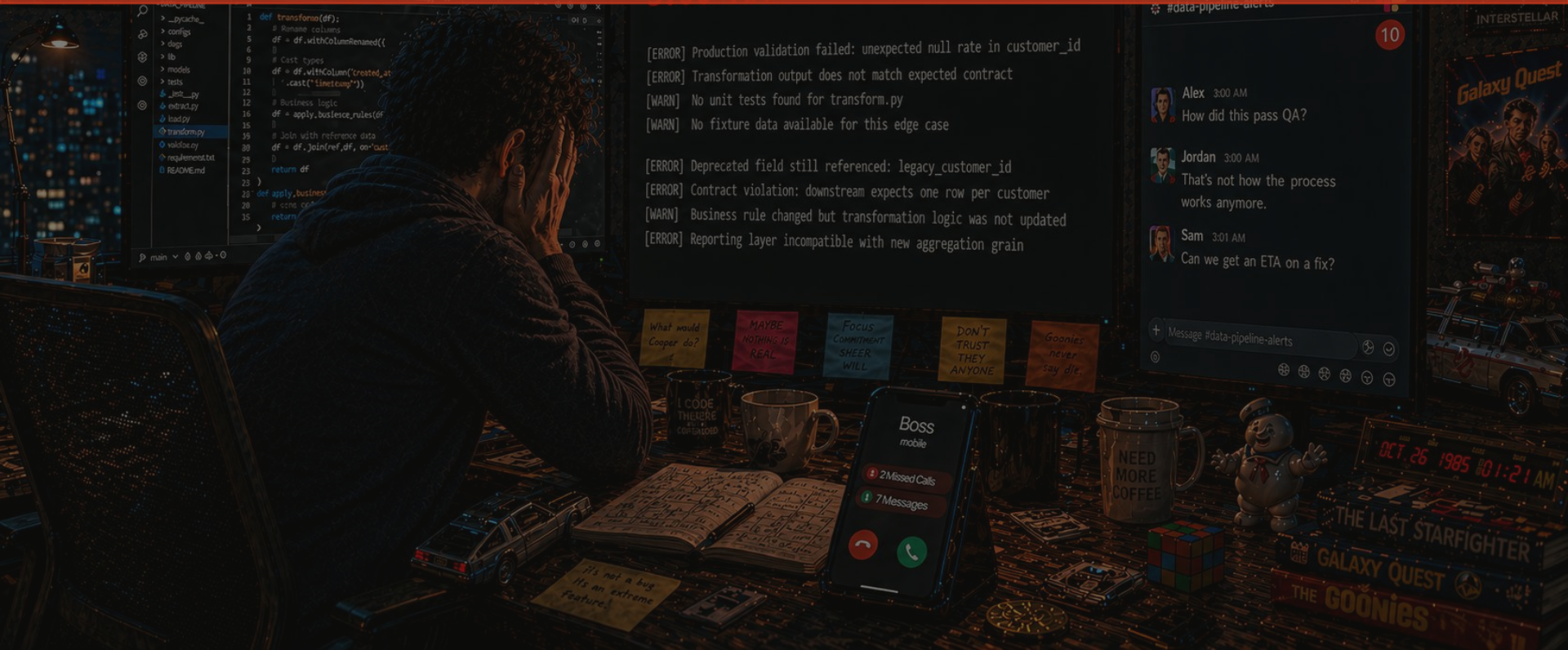
Fix 4: Test Without Consequences

```
def run():
    extract(prod_source)
    load(prod_target)
# Running the pipeline is exciting.
# People are nervous. That means it's not tested enough.
```

Fix 4: Test Without Consequences

```
def run(source, target):
    rows = extract(source)
    load(target, rows)
# End-to-end run against a safe environment — real code path, fake blast radius
run(source=staging_source, target=temp_schema("test_run_" + run_id))
assert_row_count(temp_schema, expected=10_000, tolerance=0.05)
assert_no_nulls(temp_schema, fields=["customer_id", "event_ts"])
assert_no_duplicates(temp_schema, key="id")
drop_schema(temp_schema)
```

THE PAST HAS CHANGED!




THE LAST STARFIGHTER
INCEPTION
THE GOONIES
MATRIX
GHOSTBUSTERS
OUTATIME
INTERSTELLAST
GALAXY QUEST
BACK TO THE FUTURE
3:00
DATA
SLEEP

THE REMAKE

Bruce Redeemed.

BEFORE

- Silent failures at 3am
- Doubled data on reruns
- Unable to do any meaningful performance tests or stress tests
- Explodes when schema changes
- Full rewrite when requirements shift

AFTER

- Anomaly alerts catch issues early
- Idempotent — safe to retry
- Full test suite, no consequences, we did performance and stress testing
- Schema evolution handled gracefully
- Pipeline that bends instead of breaks

BEFORE YOU GO

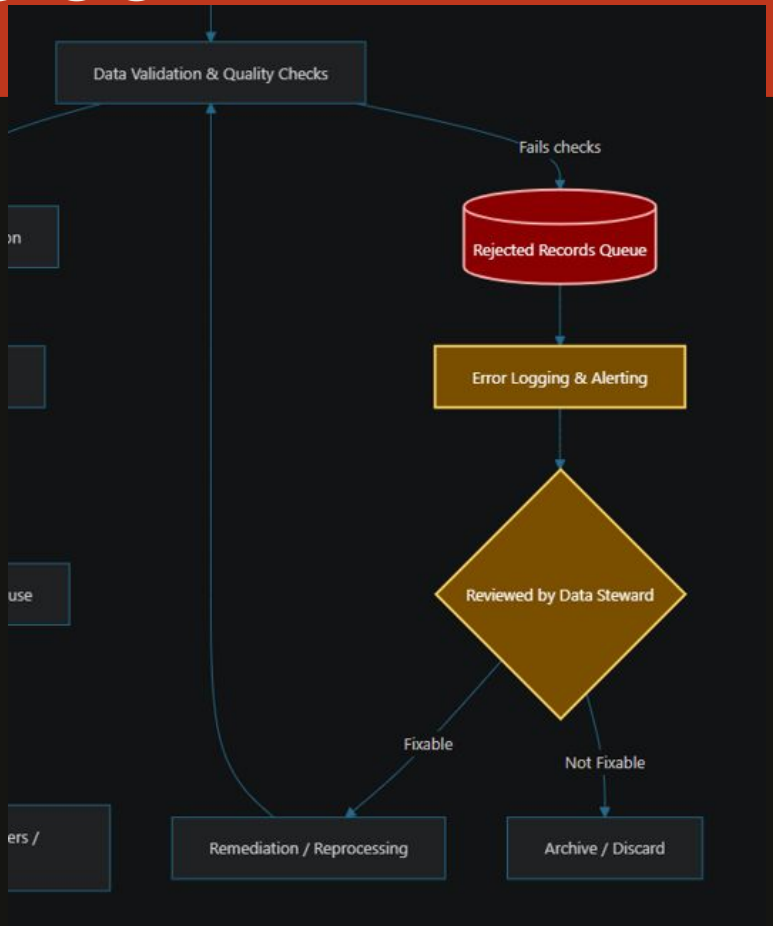
01 Build for schema change.

02 Make it idempotent.

03 Observe everything.

04 Test without consequences.

BEFORE YOU GO



Bruce left before he could fix it.

You're still here.

You can fix it!

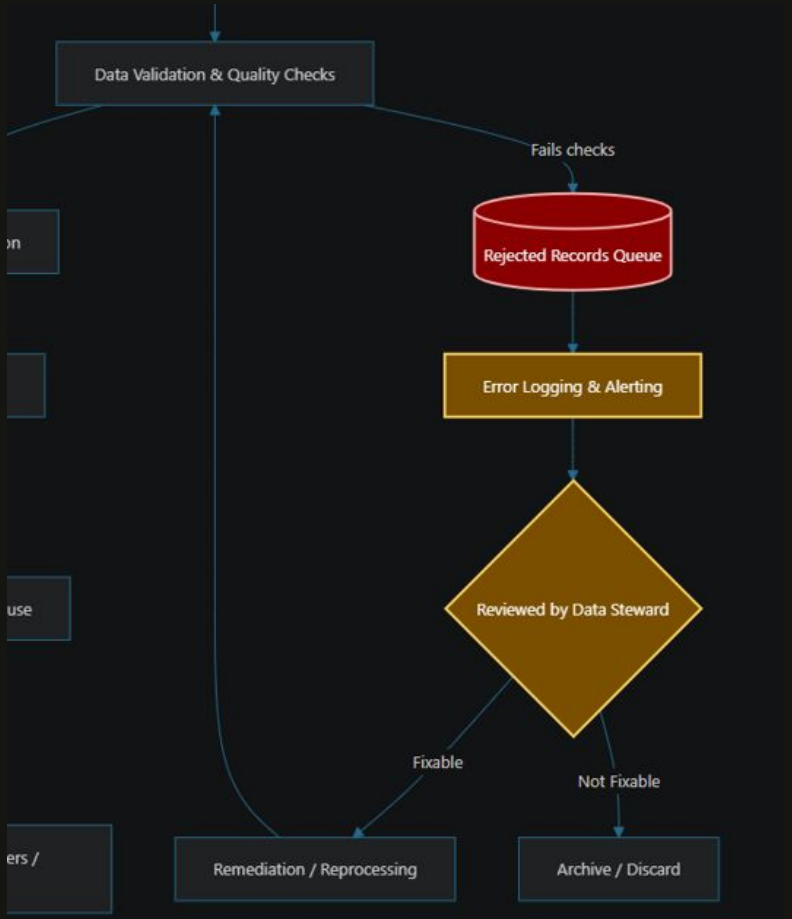
Start Small, Do Something

"Go enjoy the conference. You've already seen the real disaster film."

Chris Birie
Software & Data Martial Artist
chris.birie@gmail.com






Data Validation & Quality Checks
Fails checks
Rejected Records Queue
Error Logging & Alerting
Reviewed by Data Steward
Fixable
Not Fixable
Remediation / Reprocessing
Archive / Discard

Bruce left before he could fix it.

You're still here.

You can fix it!

Start Small, Do Something

"Go enjoy the conference. You've already seen the real disaster film."

Chris Birie
Software & Data Martial Artist
chris.birie@gmail.com



